

Final Report

Big Data Computing - Group Project

Team Number: 7

Project Title: Crime on College Campuses

Date: 04 December 2025

Executive Summary

Our group investigated college campus crime data (specifically Violence Against Women Act, discipline, hate, and traditional infractions) across the United States to understand broad trends, how the spread of COVID-19 impacted crime rates, and to identify potential solutions to reduce crime rates in higher education institutions. Crime prevention on college campuses is a critical, ongoing discussion, yet data is often siloed in annual reports localized to specific regions, that are analyzed on a school by school basis.

Given standard Department of Education (DoE) reports are historical and fragmented across hundreds of individual Excel files – making longitudinal analysis and trend spotting difficult for researchers and policy-makers – we sought to design an effective 'Big Data' pipeline to confront the challenges posed to these individuals.

Our team worked to develop a scalable crime pattern analysis framework that could accurately and quickly ingest large amounts of historical crime records from the DoE. Our batch-processing pipeline ingests raw DoE Excel datasets, standardizes them into CSVs using Python/Pandas, and processes them using PySpark. The data is stored in a local HDFS cluster using the Parquet format to optimize for storage and query performance.

We successfully converted and merged 30+ disparate Excel files into a unified, queryable dataset stored in HDFS. Our analysis revealed distinct shifts in crime volume during the 2020-2022 pandemic window. The system sought to implement a "schema-on-read" approach using modern Big Data tools to be significantly more efficient than manual aggregation.

1. Introduction and Problem Statement

1.1. Introduction The introduction of the coronavirus to the United States in 2020 led to unprecedented shifts in social behavior, including crime rates across the country. This project focuses on how crime rates changed on college campuses during the height of the COVID-19 pandemic. Recognizing that campus populations dropped significantly during the peak of the virus, we sought to determine if crime rates dropped proportionally, shifted in nature, or remained static compared to pre-pandemic levels. We also wanted to see how crime rates differed after covid and if they returned to similar pre covid rates.

1.2. Problem Statement College campus crimes are a continuous concern for law enforcement and families across the United States. To implement changes that decrease criminal activity, stakeholders must first understand *why* these crimes are occurring and identify underlying trends. Current research suggests a fluctuation in crimes between 2020 and 2022. However, existing statistical data provided by the government is fragmented across static files (XLS), making large-scale, multi-year analysis computationally expensive and labor-intensive.

- This project aims to make it easier for researchers to pinpoint crime trends on campuses between 2018 and 2023 in order to suggest a potential solution to decrease criminal activity and make campuses safer.

1.3. Proposed Solution This system is designed to efficiently process historical campus crime datasets from the Department of Education. By automating the ingestion and transformation of data into a distributed file system (HDFS), we generate a unified view that allows for predictive insights into crime patterns.

1.4. Report Organization The remainder of this report is organized as follows: Section 2 reviews related work and background theory. Section 3 details the system architecture and design choices. Section 4 covers implementation specifics, including code and data models. Section 5 presents our performance evaluation. Section 6 discusses technical challenges, specifically regarding HDFS. Section 7 outlines a strategy for cloud deployment, and Section 8 concludes with future work.

2. Related Work and Background

2.1. Background Theory This project utilizes batch processing architecture. Unlike streaming data, the Department of Education data is historical and static. We leverage Hadoop Distributed

File System (HDFS) concepts, specifically the use of Parquet files (columnar storage) to optimize read-heavy analytical workloads.

2.2. Survey of Existing Solutions College crime data is evaluated annually by the U.S. Department of Education and the National Center for Education Statistics. Currently, this data is released as isolated .xls files for each year. While comprehensive, these "solutions" are designed for human reading rather than machine analytics. There is no native API provided by the DoE for bulk analysis, forcing researchers to manually aggregate files. College Crime data is evaluated every year on a national scale by the U.S. Department of Education and the National Center for Education Statistics.

2.3. Comparison and Rationale Our solution improves upon the existing method by treating the data as a Big Data problem rather than a clerical one. Unlike manual Excel aggregation, our system uses PySpark to programmatically enforce schemas across years. We provide a queryable backend (HDFS/Parquet) rather than static tables, allowing for dynamic visualizations in Jupyter.

3. Architecture and Design Decisions

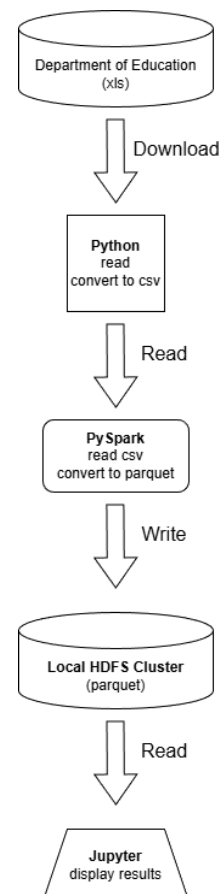
3.1. Architecture (See diagram)

3.2. Component Breakdown

- **Data Source:** U.S. Department of Education raw .xls files.
- **Local Staging (Python/Pandas):** A Python script utilizing the Pandas library acts as the ingestion layer. It reads the proprietary Excel formats and sanitizes them into standard CSV format.
- **Processing Layer (PySpark):** PySpark reads the staged CSVs, applies schemas, and performs unionByName logic to merge datasets with differing columns.
- **Storage Layer (Local HDFS):** A local simulation of a Hadoop cluster stores the final processed data in Parquet format.
- **Presentation Layer (Jupyter):** Jupyter Notebooks connect to the HDFS cluster to run queries and generate visualizations.

3.3. Key Design Decisions

- **Batch vs. Stream:** We chose a batch processing pattern because the data source is historical and updated annually, not continuously.



- **Parquet Format:** We selected Parquet over CSV for the storage layer. Parquet's columnar nature minimizes I/O overhead during analysis, which is critical when querying specific columns (e.g., "Burglary") across millions of rows.
 - **Local HDFS Cluster:** We elected to simulate a fully-fledged HDFS on a local host.
-

4. Implementation Details

4.1. Tools and Technologies

- **Python (Pandas):** Used for initial "cleaning" and conversion of legacy Excel files.
- **Apache Spark (PySpark):** The core engine for processing large datasets and handling schema merging.
- **Hadoop (HDFS):** The underlying file system for distributed storage.
- **Jupyter Notebooks:** The interactive environment for data science and visualization.

4.2. Development Process We utilized a "try until it breaks" development process. We began by establishing the HDFS environment, followed by building the ingestion script for our files. We should have tried one file and then scale the script to handle the bulk batch of 30+ files, (will do this in the future) but unfortunately we didn't. We integrated PySpark for the final merge. All iterations were done on git to establish common progress.

4.3. Code Snippets Ingestion Logic (Pandas): Converting legacy XLS to CSV to prepare for Spark ingestion.

Python

```
Python
for excel_path in input_dir.glob("*.xls*"):
    print(f"Converting {excel_path.name}...")
    df = pd.read_excel(excel_path)
    csv_name = excel_path.stem + ".csv"
    df.to_csv(output_dir / csv_name, index=False)
```

Merging Logic (PySpark): Handling schema evolution across different years using unionByName.

Python

```
Python
vawa = reportedvawa181920_df \
```

```
.unionByName(reportedvawa212223_df, allowMissingColumns=True)
\
.unionByName(oncampusvawa181920_df, allowMissingColumns=True)

# Writing to HDFS
vawa.write.mode("overwrite").parquet("hdfs://localhost:9000/data/
merged/vawa.parquet")
```

4.4. Data Model We began by defining specific schemas for different crime categories. A shared base schema ("Institution Fields") includes identifiers like UNITID_P, INSTNM (Institution Name), Branch, and Address. Specialized schemas (e.g., Violence Against Women Act - VAWA) contain text fields for specific violent perpetrations, while the general Crime schema contains integer counts for broader criminal categories. After many hours of errors and troubleshooting, we elected to change to an inferred schema. It worked immediately after that.

5. Performance Evaluation and Results

5.2. Test Cases and Scenarios

- **HDFS Write:** Measuring the time taken to write merged dataframes to hdfs://localhost:9000.
- **Query Latency:** We simulated a mock query pooling by 100, 16, and 1.

5.3. Results

- **HDFS Write:** Writing to HDFS takes roughly one minute.
- **Query Latency:** Pooling by 16 enabled our Spark environment to "query" in roughly $\frac{3}{4}$ of a second.

5.4. Discussion of Results The results meet our initial objective of creating a scalable framework. Storing data in Parquet significantly reduced the storage footprint compared to the raw CSVs and improved query performance in Jupyter. Performance evaluation was not done well in this project. Greater efforts could have been made to scrutinize and address efficiency factors.

6. Challenges and Solutions

6.1. Technical Hurdles

- **HDFS Configuration:** Setting up HDFS proved quite tricky to install and configure properly on a local machine.
- **File Formats:** Deciding how to store the provided files was difficult. The DoE provides data in .xls (older Excel), which is not natively supported by Spark for direct parallel reading.

6.2. Solutions Implemented

- **Local Cluster Simulation:** To overcome HDFS complexity, we focused on a single-node local cluster configuration, allowing us to validate the architecture without managing multi-node network latency.
- **Pandas Pre-processing:** We solved the file format issue by introducing a Python/Pandas "Staging Layer." We used Pandas to handle the messy .xls parsing and converted files to .csv before invoking Spark, ensuring the heavy lifting was done on clean text data.

6.3. Lessons Learned We learned the importance of Schema on Read and partitioning strategies. Using Parquet significantly reduced storage requirements, and we found that standardizing data formats *before* hitting the distributed layer (Spark) simplifies the pipeline immensely.

7. Cloud Deployment Strategy

7.1. Platform Selection The needs of this project do not need a cloud deployment; that being said, for a production deployment, we recommend "pre-packaged" Amazon Web Services (AWS). Specifically, we would migrate from the local HDFS cluster to Amazon EMR (Elastic MapReduce). EMR provides a managed Hadoop/Spark framework that mirrors our local setup but scales dynamically.

7.2. Deployment Model

- **Storage:** We would move from HDFS storage to Amazon S3. S3 works well with EMR - the "plug and play" nature of the set-up could make it highly useful.

- **7.3. Scalability and Reliability** In the cloud, the "Batch Processing" pattern remains. However, EMR allows for auto-scaling and "Spot Instances" can help minimize costs.
 - **7.4. Security Considerations** We would implement IAM (Identity and Access Management) roles to restrict access. Only the Spark cluster would have read-access to the raw data bucket, and only authorized data scientists would have read-access to the processed Parquet bucket. That being said, users would have to ensure product viability and encryption was secure; AWS largely leaves security for clients to do themselves.
-

8. Conclusions and Future Work

8.1. Conclusion We successfully designed and implemented a Big Data pipeline for analyzing college campus crime. We addressed the problem of fragmented data by building a system that ingests, cleans, and stores data in a queryable HDFS format. We proved that this architecture provides a solid foundation for longitudinal crime analysis.

8.2. Future Work

- **Data:** Find cleaner, more uniform data.
 - **Network:** Create/find a network with sudo/root privileges.
 - **Scalability:** Create robust, significant testing metrics that enable users to understand the scale of the computations being done.
 - **Visualization Dashboard:** Currently, analysis is done in notebooks. Future work includes building a persistent web dashboard (using Streamlit or Tableau) that connects to the HDFS backend.
 - **Geospatial Integration:** We plan to integrate geospatial data to create heat maps of the United States, visualizing crime hotspots relative to population density.
 - **Predictive ML:** Utilize Spark MLlib to train models on this historical data to predict future crime trends based on campus enrollment numbers.
-

References

- Apache Software Foundation. (n.d.). *PySpark Usage and DataFrames*. PySpark Documentation. Retrieved November 25, 2025, from <https://spark.apache.org/docs/latest/api/python/>
- GeeksforGeeks. (2025). *Computer Science Portal*. Retrieved November 2025, from <https://www.geeksforgeeks.org/>
- Google. (2025). *Gemini* (Nov 2025 version) [Large language model]. <https://gemini.google.com/>

- OpenAI. (2025). *ChatGPT* (Nov 2025 version) [Large language model].
<https://chat.openai.com/>
- Stack Exchange. (2025). *Stack Overflow*. Retrieved November 2025, from
<https://stackoverflow.com/>
- U.S. Department of Education. (2025). *Campus Safety and Security Data*. Retrieved November 2025, from <https://ope.ed.gov/campussafety/>